

Algorithmique 1 — TP 1

Environnement de TP & tests unitaires · UE Info1.Algo1 · L1 Informatique, UPS · Semestre 1
2026-2027

Les exercices sont présentés en deux temps : **Partie 1 — Énoncés** (le code fourni et les tâches demandées, sans réponse, pour pouvoir refaire le TP) puis, après un saut de page, **Partie 2 — Corrigé** (raisonnement complet, correction et jeux de test vérifiés).

Organisation et environnement de travail

Chaque TP est composé d'un fichier PDF (énoncé) et d'un ou plusieurs fichiers Python à compléter pendant la séance, téléchargeables sur Moodle (onglet « TP »). Il faut organiser ses fichiers dans des répertoires cohérents (répertoire dédié si travail sur machine de l'université). Une zone de dépôt existe pour son propre confort mais le dépôt n'est pas obligatoire et n'est pas corrigé par l'encadrant·e.

- **CC3 et CC4 se déroulent sur une machine de l'université**, sous un environnement Windows spécifique qui met à disposition IDLE, VS Code et PyCharm — aucun autre outil.
- Il faut donc se familiariser avec l'un de ces trois environnements **pendant les séances de TP**, pas seulement utiliser Basthon en ligne.

Environnements disponibles

Environnement	Système	Usage
IDLE	Windows & Linux	Clic droit sur le fichier → <i>Edit with IDLE</i> , puis menu <i>Run</i> ou touche F5 pour exécuter. Solution la plus simple, suffit pour tous les TP de l'UE.
VS Code	Windows & Linux	Clic droit → <i>Edit with VS Code</i> . Au premier lancement, accepter Trust si un bandeau « Restricted Mode » apparaît, puis lancer avec le bouton Run Python File .
PyCharm	Windows uniquement	Fonctionne par projet (pas par fichier isolé) : créer un « Projet Pure Python » avec un interpréteur Python 3.x récent, déposer les fichiers du sujet dans un sous-répertoire (ex. TP1). En cas d'erreur au <i>Run</i> , vérifier l'interpréteur associé au projet.

Sous Linux, seuls IDLE et VS Code sont disponibles ; leur usage est identique à celui décrit sous Windows.

Travail personnel

Une version de **Python 3** récente suffit, avec l'environnement de son choix (VS Code, PyCharm, Emacs...). Sans possibilité d'installation : **Basthon** (`console.basthon.fr`) fonctionne pour tous les TP, **sauf celui consacré à la complexité** qui nécessite un environnement local.

Rappels — principe des tests unitaires

- Un **jeu de test** spécifie, pour une fonction et des paramètres donnés, la valeur de retour ou l'**effet** attendu (cas d'une liste modifiée en place).
- Les jeux de test doivent couvrir le **maximum de cas possibles** : cas particuliers et cas généraux.
- L'objectif est de **mettre en défaut** le code testé, pas seulement de vérifier qu'il marche sur un exemple facile.

Syntaxe d'une assertion

```
assert <expression booléenne>, "message d'erreur"
```

Sémantique : le message d'erreur est affiché seulement si l'expression booléenne vaut `False` ; sinon l'exécution est silencieuse.

Convention de nommage : les jeux de test d'une fonction `f` sont rassemblés dans une fonction de test `test_f`.

Partie 1 — Énoncés

Exercice 1 — palindrome ★ (facile)

ex01_palindrome.py fournit est_palindrome (True si la chaîne est un palindrome) et sa fonction de test. La fonction est mal conçue et le jeu de test ne le révèle pas.

```
def est_palindrome(mot):
    debut = 0
    fin = len(mot) - 1
    while debut != fin:
        if not mot[debut] == mot[fin]:
            return False
        debut += 1
        fin -= 1
    return True
```

- Ajouter au jeu de test l'assertion `assert est_palindrome('abba')==True` et exécuter le fichier. Que se passe-t-il ?
- Corriger la fonction pour qu'elle fonctionne sur ce cas.
- Valider la correction avec la chaîne vide et un contre-exemple non-palindrome de longueur paire.

Exercice 2 — maximum ★ (facile)

ex02_maximum.py fournit maximum (retourne le plus grand élément d'une liste non vide d'entiers) et sa fonction de test, mal conçue : il manque un cas qui révélerait une erreur.

```
def maximum(liste):
    m = 0
    for e in liste:
        if e>m:
            m = e
    return m
```

- Trouver un jeu de valeurs qui révèle une erreur dans la fonction, et l'ajouter au test.
- Corriger la fonction (une modification minimale, une seule ligne, suffit).

Exercice 3 — répétitions consécutives ★ (facile)

ex03_repetitions_consecutives.py : `eliminer_repetitions_consecutives` remplace dans la liste passée en paramètre toute séquence de 2 entiers identiques consécutifs (ou plus) par un seul — effet sur la liste, retourne None. Le jeu de test est incomplet.

```
def eliminer_repetitions_consecutives(liste):
    i = 1
    while i<len(liste):
        if liste[i]==liste[i-1]:
            liste.pop(i)
            i += 1
```

- Trouver un cas non couvert par les tests fournis qui révèle une erreur (indice : le nombre d'occurrences identiques consécutives).
- Corriger la fonction en ne modifiant que la structure de contrôle, sans ajouter de ligne de calcul.

Exercice 4 — occurrences ★ (facile)

ex04_occurrences.py fournit nb_occurrences(liste, valeur), qui retourne le nombre d'occurrences de valeur dans liste. **La fonction n'est pas à réécrire** : il s'agit uniquement de compléter test_nb_occurrences avec un jeu de test couvrant les 9 cas ci-dessous.

La fonction affiche, en plus du calcul, une grille à 9 colonnes qui indique quel cas vient d'être reconnu (une croix par colonne). Il faut choisir un exemple pour chacune des 9 situations, sans jamais retomber deux fois sur la même colonne :

Cas à couvrir

liste vide

un seul élément = valeur

un seul élément $\neq$ valeur

$n \geq 5$, valeur absente

$n \geq 5$, une occurrence en début

$n \geq 5$, une occurrence en fin

$n \geq 5$, une occurrence ni en début ni en fin

$n \geq 5$, plusieurs occurrences + autres valeurs

$n \geq 5$, uniquement la valeur

Exercice 5 — années bissextiles (boîte noire) ★ (facile)

Test en **boîte noire** : on ne regarde pas le code, seulement la spécification. `est_bissextile(annee)` doit être vraie si l'année est divisible par 4 mais pas par 100, ou si elle est divisible par 400.

- Identifier les 4 cas à couvrir, d'après les critères de divisibilité par 4 et par 100.
- Compléter `test_est_bissextile` avec un exemple pour chacun des 4 cas.

Exercice 6 — stricte croissance (boîte blanche) ★ (facile)

Test en **boîte blanche** : cette fois on choisit les cas en regardant le **code source**, pour couvrir les différents chemins d'exécution possibles. `est_strictement_croissante(liste)` retourne True si la liste est strictement croissante.

```
def est_strictement_croissante(liste):
    i = 0
    resultat = True
    while resultat and i < len(liste) - 1:
        if liste[i] >= liste[i + 1]:
            resultat = False
        i += 1
    afficher_croix(i, resultat)
    return resultat
```

- Écrire 7 cas de test, croisés sur la valeur de retour (True/False) et le nombre d'itérations de la boucle while (`afficher_croix` et la fonction elle-même ne doivent pas être modifiées).
- Expliquer pourquoi un cas « False avec 0 itération » n'existe pas.

Exercice 7 — deux miroirs ★★ (intermédiaire)

Objectif : écrire une fonction miroir d'une liste (éléments dans l'ordre inverse), sans utiliser `[::-1]` ni `.reverse()` — uniquement des boucles avec accès par indice et `.append()` / `.pop()` / `.insert()`. Deux fonctions sont demandées, à distinguer grâce à leurs tests respectifs :

- `miroir_A` : les tests vérifient `liste == liste` après l'appel (la liste d'origine est intacte) et que la valeur retournée est la liste inversée.
- `miroir_B` : les tests vérifient que l'appel retourne `None` et que c'est la liste passée en paramètre qui se retrouve inversée.

Écrire `miroir_A` et `miroir_B` en fonction de ce que chaque jeu de test impose.

Exercice 8 — fonctions mystères ★★★ (avancé)

Aucun énoncé fonctionnel n'est donné : il faut deviner ce que font `mystere_1(liste, n)` (retourne une nouvelle liste, n'y touche pas) et `mystere_2(liste, n)` (même résultat, mais par effet de bord, sans retour) uniquement à partir des assertions du fichier fourni :

```
liste = [9,5,10]
assert mystere_1(liste,0) == [9,5,10]
assert mystere_1(liste,1) == [5,10,9]
assert mystere_1(liste,2) == [10,9,5]
assert mystere_1(liste,3) == [9,5,10]

liste2 = [8,5,6,13,2,8]
assert mystere_1(liste2,8) == [6,13,2,8,8,5]
assert mystere_1(liste2,-666) == [8,5,6,13,2,8]
```

- En déduire, à partir de ces assertions, ce que calculent `mystere_1` et `mystere_2`.
- Implémenter les deux fonctions.

Partie 2 — Corrigé

Exercice 1 — palindrome ★ (facile)

```
def est_palindrome(mot):
    debut = 0
    fin = len(mot) - 1
    while debut != fin:
        if not mot[debut] == mot[fin]:
            return False
        debut += 1
        fin -= 1
    return True
```

1) Ajout du test 'abba' : `assert est_palindrome('abba')==True`. On constate une **erreur à l'exécution** (index hors limites), jamais un simple `False`. Explication : à chaque tour, `debut` augmente de 1 et `fin` diminue de 1, donc l'écart `debut - fin` varie toujours de 2 en 2. Pour une chaîne de longueur **paire**, cet écart part d'une valeur **impaire** (ex. -3 pour 'abba') : il ne peut donc jamais valoir 0, et la condition `debut != fin` reste vraie indéfiniment — les indices continuent de se croiser et sortent des bornes du mot. La boucle ne s'arrête correctement que pour les longueurs impaires (écart initial pair).

2) Correction : remplacer la condition d'arrêt par une comparaison qui reste valable même quand les indices se croisent :

```
def est_palindrome(mot):
    debut = 0
    fin = len(mot) - 1
    while debut < fin:
        if not mot[debut] == mot[fin]:
            return False
        debut += 1
        fin -= 1
    return True
```

3) Validation avec la chaîne vide (trivialement palindrome : la boucle ne s'exécute jamais) et un contre-exemple pair non-palindrome :

```
assert est_palindrome('abba')==True
assert est_palindrome('')==True
assert est_palindrome('abcdba')==False
```

Exercice 2 — maximum ★ (facile)

```
def maximum(liste):
    m = 0
    for e in liste:
        if e > m:
            m = e
    return m
```

1) Cas oublié : une liste ne contenant **que des nombres négatifs**. L'accumulateur est initialisé à `m = 0`, qui n'est jamais dépassé par un nombre négatif : la fonction renverrait 0 au lieu du vrai maximum.

```
assert maximum([-5, -2, -8]) == -2
```

2) Correction minimale (une seule ligne) : initialiser l'accumulateur avec le premier élément de la liste plutôt qu'avec 0 :

```
def maximum(liste):
    m = liste[0]
    for e in liste:
        if e > m:
            m = e
    return m
```

Exercice 3 — répétitions consécutives ★ (facile)

```
def eliminer_repetitions_consecutives(liste):
    i = 1
    while i < len(liste):
        if liste[i] == liste[i-1]:
            liste.pop(i)
            i += 1
```

1) Cas oublié : une séquence de **trois occurrences identiques ou plus** (tous les cas fournis n'en ont que deux à la fois). Exemple : `[1, 5, 5, 5, 2]` devrait donner `[1, 5, 2]`.

```
liste = [1, 5, 5, 5, 2]
assert eliminer_repetitions_consecutives(liste) == None and liste == [1, 5, 2]
```

Avec le code d'origine, ce cas échoue : après un `pop(i)`, l'indice `i` est quand même incrémenté, alors que l'élément qui vient de glisser en position `i` n'a pas été recomparé. Pour `[1, 5, 5, 5, 2]` : un seul doublon est éliminé, on obtient `[1, 5, 5, 2]` au lieu de `[1, 5, 2]`.

2) Correction minimale (structure de contrôle uniquement, sans ligne de calcul supplémentaire) : ne faire avancer `i` que lorsqu'il n'y a **pas** eu de suppression, via un `else` :

```
def eliminer_repetitions_consecutives(liste):
    i = 1
    while i < len(liste):
        if liste[i] == liste[i-1]:
            liste.pop(i)
        else:
            i += 1
```

Exercice 4 — occurrences ★ (facile)

Cas demandé	Exemple utilisé	Résultat attendu
liste vide	<code>([], 5)</code>	0
un seul élément = valeur	<code>([5], 5)</code>	1
un seul élément ≠ valeur	<code>([3], 5)</code>	0
$n \geq 5$, valeur absente	<code>([1, 2, 3, 4, 6], 5)</code>	0
$n \geq 5$, une occurrence en début	<code>([5, 1, 2, 3, 4], 5)</code>	1
$n \geq 5$, une occurrence en fin	<code>([1, 2, 3, 4, 5], 5)</code>	1
$n \geq 5$, une occurrence ni en début ni en fin	<code>([1, 2, 5, 3, 4], 5)</code>	1
$n \geq 5$, plusieurs occurrences + autres valeurs	<code>([5, 1, 5, 2, 3], 5)</code>	2
$n \geq 5$, uniquement la valeur	<code>([5, 5, 5, 5, 5], 5)</code>	5

```
def test_nb_occurrences():
    print('Test de la fonction nb_occurrences')
    assert nb_occurrences([], 5) == 0
    assert nb_occurrences([5], 5) == 1
    assert nb_occurrences([3], 5) == 0
    assert nb_occurrences([1, 2, 3, 4, 6], 5) == 0
    assert nb_occurrences([5, 1, 2, 3, 4], 5) == 1
    assert nb_occurrences([1, 2, 3, 4, 5], 5) == 1
    assert nb_occurrences([1, 2, 5, 3, 4], 5) == 1
    assert nb_occurrences([5, 1, 5, 2, 3], 5) == 2
    assert nb_occurrences([5, 5, 5, 5, 5], 5) == 5
    print(' OK')
```

Exercice 5 — années bissextiles (boîte noire) ★ (facile)

1) Les 4 cas à couvrir, d'après les critères de divisibilité par 4 et 100 :

Cas	Exemple	Bissextilé ?
non divisible par 4	2025	Non
divisible par 4, pas par 100	2024	Oui
divisible par 4 et par 100, pas par 400	1900	Non
divisible par 400	2000	Oui

2) Complément de test_est_bissextilé :

```
def test_est_bissextilé():
    print('Test de la fonction est_bissextilé')
    assert est_bissextilé(2025)==False
    assert est_bissextilé(2024)==True
    assert est_bissextilé(1900)==False
    assert est_bissextilé(2000)==True
    print(' OK')
```

Le fichier fournit une fonction `est_bissextilé` volontairement **obfusquée** (illisible) mais correcte : elle encode le même calcul via une unique expression à base de fonctions lambda qui s'appellent elles-mêmes, plutôt qu'avec des `def` et des `if` classiques. Ce n'est pas demandé de la comprendre ni de la réécrire : le principe du test en boîte noire est justement de valider une fonction sans lire son code, seulement sa spécification.

Exercice 6 — stricte croissance (boîte blanche) ★ (facile)

```
def est_stricte_croissance(liste):
    i = 0
    resultat = True
    while resultat and i<len(liste)-1:
        if liste[i]>=liste[i+1]:
            resultat = False
        i += 1
    afficher_croix(i,resultat)
    return resultat
```

1) Il faut 7 cas, croisés sur la valeur de retour et le nombre d'itérations de la boucle while :

Retour	Itérations	Exemple	Pourquoi
True	0	[4]	liste d'un seul élément : la boucle ne tourne jamais
True	1	[1, 2]	une seule comparaison, qui réussit
True	2	[1, 2, 3]	deux comparaisons, toutes réussies
True	>3	[1, 2, 3, 4, 5]	4 comparaisons toutes réussies
False	1	[5, 3]	la 1re comparaison échoue, la boucle s'arrête aussitôt
False	2	[1, 5, 3]	1re comparaison ok, 2e échoue
False	>3	[1, 2, 3, 4, 2]	3 comparaisons ok, la 4e échoue

```
def test_est_stricte_croissance():
    print('Test de la fonction est_stricte_croissance')
    assert est_stricte_croissance([4])==True
    assert est_stricte_croissance([1, 2])==True
    assert est_stricte_croissance([1, 2, 3])==True
    assert est_stricte_croissance([1, 2, 3, 4, 5])==True
    assert est_stricte_croissance([5, 3])==False
    assert est_stricte_croissance([1, 5, 3])==False
    assert est_stricte_croissance([1, 2, 3, 4, 2])==False
    print(' OK')
```

2) Pourquoi n'existe-t-il pas de cas « False avec 0 itération » ? Parce que c'est structurellement impossible : `resultat` vaut `True` à l'initialisation et ne peut devenir `False` qu'à l'intérieur de la boucle. S'il y a 0 itération, le corps de la boucle ne s'exécute jamais, donc `resultat` reste forcément à sa valeur initiale `True`. Ce cas n'est pas seulement « oublié », il est **inatteignable**.

Exercice 7 — deux miroirs ★★ (intermédiaire)

`miroir_A` : on construit une liste neuve en parcourant l'originale à l'envers par indice, sans jamais la modifier :

```
def miroir_A(liste):
    resultat = []
    n = len(liste)
    for i in range(n):
        resultat.append(liste[n-1-i])
    return resultat
```

`miroir_B` : on échange les éléments symétriques deux à deux, directement dans la liste (lecture/écriture par indice, autorisée par l'énoncé) :

```
def miroir_B(liste):
    n = len(liste)
    for i in range(n//2):
        liste[i], liste[n-1-i] = liste[n-1-i], liste[i]
```

`n//2` échanges suffisent : pour une liste de longueur impaire, l'élément du milieu reste à sa place (il est son propre miroir), aucun traitement particulier n'est nécessaire.

Exercice 8 — fonctions mystères ★★★ (avancé)

Déduction : pour `liste = [9, 5, 10]`, on observe `mystere_1(liste, 0) == [9, 5, 10]`, `mystere_1(liste, 1) == [5, 10, 9]`, `mystere_1(liste, 2) == [10, 9, 5]`, `mystere_1(liste, 3) == [9, 5, 10]` (retour au point de départ), et des valeurs négatives ou très grandes de `n` donnent toujours un résultat cohérent avec `n modulo la longueur de la liste`. C'est exactement une **rotation vers la gauche de n positions** : les `n % taille` premiers éléments passent à la fin.

Vérification sur le plus grand exemple (`[8, 5, 6, 13, 2, 8]`, taille 6) : `n=8 → 8%6=2 → rotation de 2 → [6, 13, 2, 8, 8, 5]` (conforme au test) ; `n=-666 → -666%6=0 → liste inchangée (conforme)`.

`mystere_1` — nouvelle liste, sans modifier l'originale :

```
def mystere_1(liste, n):
    taille = len(liste)
    if taille == 0:
        return []
    d = n % taille
    return liste[d:] + liste[:d]
```

`mystere_2` — même rotation, mais par effet de bord : on retire les `d` premiers éléments et on les remet à la fin, sur l'objet liste lui-même :

```
def mystere_2(liste, n):
    taille = len(liste)
    if taille == 0:
        return
    d = n % taille
    partie = liste[:d]
    del liste[:d]
    liste.extend(partie)
```

Chaque fonction ne contient qu'un seul `if` (le cas dégradé de la liste vide, pour éviter une division par zéro). L'énoncé signale qu'une version **sans aucun** `if` existe : il suffit de remplacer `taille` par `len(liste) or 1` dans le calcul de `d` (astuce de court-circuit : si la liste est vide, `len(liste)` vaut 0, donc `or 1` le remplace par 1 ; le résultat reste une liste vide de toute façon).

À retenir

- **Boîte noire vs boîte blanche** : une boîte noire (ex. 5) choisit ses cas depuis la spécification, sans lire le code ; une boîte blanche (ex. 6) choisit ses cas depuis les chemins d'exécution du code source.
- **Trois bugs corrigés dans ce TP** : palindrome (condition d'arrêt de boucle qui ne gère pas les longueurs paires), maximum (accumulateur initialisé à 0 au lieu du premier élément, invalide si tous les nombres sont négatifs), répétitions consécutives (indice incrémenté même après une suppression, ce qui saute un élément lors de 3 occurrences ou plus).
- **Ex. 4, 5, 6** : il ne fallait pas réécrire les fonctions testées, seulement compléter les fonctions de test avec les bons cas.
- **Ex. 7 et 8** : toujours distinguer une fonction qui retourne une nouvelle valeur sans toucher au paramètre, d'une fonction qui modifie en place et retourne None — c'est la forme des tests qui le révèle.
- **Environnement** : s'entraîner avec IDLE, VS Code ou PyCharm dès les TP, car Basthon ne sera pas disponible pour CC3/CC4 (machine de l'université imposée) ni pour le TP sur la complexité.